

7 Arithmetic, computability, and incompleteness

The standard mathematical logic curriculum, as well as looking at elementary results about formalized theories and their models in general, investigates two particular instances of nontrivial, rigorously formalized axiomatic systems. First, there's *arithmetic* (a paradigm theory about finite whatnots), and then there is *set theory* (a paradigm theory about infinite whatnots). We consider set theory in the next chapter. This chapter is about arithmetic and related matters. More specifically:

§7.1 makes a quick general point about why the idea of computability is of such central concern to formal logic.

§7.2 introduces both the informal idea of a computable arithmetical function and the formal idea of a recursive function: the Church-Turing thesis claims that the two notions in fact pick out the same class of functions.

§7.3 introduces formal theories of arithmetic, in particular first-order Peano Arithmetic.

§7.4 outlines a version of Gödel's epoch-making first incompleteness theorem.

§7.5 gives some main reading recommendations on computability, formal arithmetic, and Gödel's theorems.

§7.6 suggests some parallel and additional reading.

§7.7 touches very briefly on some of the fascinating history of our topics (and notes a couple of annotated translations of Gödel's classic paper on his incompleteness theorems).

7.1 Logic and computability

(a) Why do we regiment informal arguments and theories into formalized versions? To eliminate ambiguities and to make everything determinate and transparently clear (even if it doesn't always seem that way to beginners!). So, for example, we want it to be entirely clear what counts as a sentence of a given theory, what counts as an axiom of the theory, and what counts as a proof in the theory. The aim of formalization is to settle these things in a way which leaves absolutely no room for doubt or dispute.

(b) As a step towards sharpening this thought, let's say as an initial rough characterization:

A property P is *effectively decidable* if and only if there is an *algorithm* (a finite set of instructions for a deterministic computation) for settling in a finite number of steps whether a relevant object has property P .

Relatedly, the answer to a question Q is effectively decidable if and only if there is an algorithm which gives the answer, again by a deterministic computation, in a finite number of steps.

To put it in only slightly different words, a property P is effectively decidable just when there's a step-by-step mechanical routine for settling whether an object of the relevant kind has property P , such that a suitably programmed deterministic computer could in principle implement the routine (idealizing away from practical constraints such as running time, etc.). Similarly, the answer to a question Q is effectively decidable just when a suitably programmed computer could deliver the answer (in principle, in finite time).

And the point we made in (a) now comes to this. In formalizing a theory we will want, for example, to be able to effectively decide whether a given string of symbols has the property of being a well-formed formula of the theory's formal language, whether a formula is an axiom of the given formal theory, and whether a sequence of formulas is a correctly formed proof of the theory. In other words, we will want to set up a formal deductive theory so that a computer could, in principle, mindlessly check the credentials of a purported proof by deciding whether each step of the proof is indeed in accordance with the official rules of the theory.

(c) NB: It is one thing to be able to effectively decide whether a purported proof of P really is a proof in a given formal theory T . It is another thing entirely to be able to effectively decide in advance whether P actually *has* a proof in T .

You'll soon enough find out that, e.g., in a properly set up formal theory of arithmetic T , we can effectively check whether a supposed proof of P in fact conforms to the rules of the game. But once we are dealing with an even mildly interesting arithmetic theory T , there will be no way of deciding in advance whether a T -proof of P exists.

It is of course nice when a theory *is* decidable, i.e. when a computer can tell us whether a given proposition does or doesn't follow from the theory. But few interesting theories are decidable in this sense; so mathematicians aren't going to be put out of business!

(d) Now, in our initial rough definition of the notion of effective decidability, we invoked the idea of what an idealized computer could (in principle) do by implementing some algorithm. This idea surely needs further elaboration.

1. As a preliminary step, we can narrow our focus and just consider the decidability of *arithmetical* properties.

Why? Because we can always represent facts about finite whatnots like formulas and proofs by using numerical codings. We can then trade in questions about formulas or proofs for questions about their code numbers.

2. And as a second step, we can also trade in questions about the effective decidability of arithmetical *properties* for questions about the algorithmic computability of numerical *functions*.

Why? Because for any property of natural numbers P we can define a corresponding numerical function (its so-called ‘characteristic function’) c_P such that if n has the property P , $c_P(n) = 1$ and if n doesn’t have the property P , $c_P(n) = 0$. Think of ‘1’ as coding for truth, and ‘0’ for falsehood. Then the question (i) ‘can we effectively decide whether a number has the property P ?’ becomes the question (ii) ‘is the numerical function c_P effectively computable by an algorithm?’.

By these two steps, we can quickly move from e.g. the question whether it is effectively decidable whether a string of symbols is a wff to a corresponding question about whether a certain numerical function is computable. So our main focus here will now be the notion of a computable numerical function..

7.2 Computable functions

- (a) Consider the following pairs of equations:

$$x + 0 = x$$

$$x + Sy = S(x + y)$$

$$x \times 0 = 0$$

$$x \times Sy = (x \times y) + x$$

$$x^0 = S0$$

$$x^{Sy} = (x^y \times x)$$

where, as in §5.2, Sx denotes the successor of x . In some notation or other, these pairs of equations should be very familiar: they in turn define addition, multiplication and exponentiation for the natural numbers.

At the risk of labouring the obvious, let’s spell out the point. Take the initial pair of equations. The first of them fixes the result of adding zero to a given number. The second fixes the result of adding the successor of y in terms of the result of adding y . Hence the two equations, applied and re-applied, tell us how to add $0, S0, SS0, SSS0, \dots$, i.e. they tell us how to add *any* natural number to a given number x . Similarly, the first of the equations for multiplication fixes the result of multiplying a given number x by zero. The second equation fixes the result of multiplying by Sy in terms of the result of multiplying by y and doing an addition. Hence the two pairs of equations together tell us how to multiply a given number

x by any of $0, S0, SS0, SSS0, \dots$. Similarly of course for the pair of equations for exponentiation.

And now note that the six equations taken together not only *define* exponentiation, but they do so by giving us an *algorithm* for computing x^y for any natural numbers x, y – they tell us how to compute x^y by doing repeated multiplications, which we in turn compute by doing repeated additions, which we compute by repeated applications of the successor function. That is to say, the chain of equations amounts to a set of instructions for a deterministic step-by-step computation which will output the value of x^y in a finite number of steps. Hence, exponentiation is an effectively computable function.

(b) In each of our pairs of equations, the second one fixes the value of the defined function for argument Sy by invoking the value of the *same* function for argument y . A procedure in which we evaluate a function for one input by calling the *same* function for some smaller input(s) is standardly termed ‘recursive’ – and the particularly simple type of procedure we’ve illustrated three times is called, more precisely, primitive recursion.

Now – arm-waving more than a bit! – consider *any* function which can be defined by a chain of equations similar to the chain of equations giving us a definition of exponentiation. Suppose that, starting from trivial functions like the successor function, we can build up the function’s definition by using primitive recursion and ‘composition’ (where we plug one function we already know about into another). Such a function is said to be *primitive recursive*.

And generalizing from the case of exponentiation, we have the following observation:

Any primitive recursive function is effectively computable.

(c) So far, so good. However, it is easy to show that

Not all effectively computable functions are primitive recursive.

A neat abstract argument proves the point.¹

But this raises an obvious question: what further ways of defining functions – in addition to primitive recursion – also give us effectively computable functions?

Here’s a pointer. The definition of (say) x^y by primitive recursion in effect tells us to start from x^0 , then loop round applying the recursion equation to compute x^1 , then x^2 , then x^3 , $\dots$, keeping going until we reach x^y . In all, we have to loop around y times. In some standard computer languages, implementing this procedure involves

¹ Roughly, we can effectively list off the primitive recursive functions by listing their defining recipes; so we have an algorithm which gives us f_n , the n -th such function. Then define the function d by putting $d(n) = f_n(n) + 1$. Evidently, d differs from any f_n for the value n , so it isn’t one of the primitive recursive functions. But it is computable.

using a ‘for’ loop (which tells us to iterate some procedure, counting as we go, and to do this for cycles numbered 1 to y). In this case, *the number of iterations is given in advance as we enter the loop*. But of course, standard computer languages also have programming structures which implement unbounded searches – they allow open-ended ‘do until’ loops (or equivalently, ‘do while’ loops). In other words, they allow some process to be iterated until a given condition is satisfied, *where no prior limit is put on the number of iterations to be executed*.

This suggests that one way of expanding the class of computable functions beyond the primitive recursive functions will be to allow computations employing *open-ended searches*. So let’s suppose we do this. There’s a standard device for implementing such searches, using a ‘minimization’ operator. Suppose we can compute whether x satisfies the condition Fx : then, roughly speaking, $\mu x Fx$ sets us off on a search through increasing values of x and returns the least x which satisfies the condition Fx . Let’s not worry about the details now; though note that since there might not be a value of x which satisfies Fx , the minimization operator may not return a value, so using it may only define a partial function. However, those *total* functions which can be computed by a chain of applications of primitive recursion and/or open-ended searches implemented by the minimization operator are called (general) *recursive*.

(d) Predictably enough, the next question is: have we *now* got all the effectively computable total functions?²

The claim that the general recursive functions are indeed just the intuitively computable total functions is *Church’s Thesis*, and is widely believed to be true (or at the very least, it is taken to be an entirely satisfactory working hypothesis). Why? For a start, there are quasi-empirical reasons: no one has found a function which is incontrovertibly computable by a deterministic algorithmic procedure but which isn’t recursive. But there are also more principled reasons for accepting the Thesis.

Consider, for example, Alan Turing’s approach to the notion of effective computation. He famously aimed to analyse the idea of a step-by-step computation procedure down to its very basics, which led him to the concept of computation by a Turing machine (a minimalist computer). And what we can call *Turing’s Thesis* is the claim that the effectively computable (total) functions are just the functions which are computable by some suitably programmed Turing machine.

So do we now have two *rival* claims, Church’s and Turing’s, about the class of numerical functions which are computable by some step-by-step algorithm? Not at all! For it turns out to be quite easy to prove the technical mathematical result that a function is recursive if and only if it is Turing computable. And so it goes: every other attempt to give an exact characterization of the class of effectively computable

² Technical note: this time, we can’t ‘diagonalize out’, because we can’t mechanically list off the recipes involving those open-ended searches that will always terminate and so define total functions.

functions turns out to locate just the *same* class of functions. That's rather remarkable, and this is one key theme you will want to explore in a first encounter with the theory of computable functions.

(e) It is fun to find out more about Turing machines, about how to effectively enumerate them, and even to learn to write a few elementary programs for them (in effect, this is learning to write in a 'machine code'). And there is a beautiful early result that you will soon encounter:

There is no mechanical decision procedure which can determine whether Turing machine number e , fed a given input n , will ever halt its computation (so there is no general decision procedure which can tell whether Turing machine e in fact computes a total function).

How do we show that? Why does it matter? I leave it to you to read up on the 'undecidability of the halting problem', and its many weighty implications. But for one example, the undecidability of the halting problem implies that there is no algorithm which can decide whether a given wff is a theorem of first-order logic.

The elementary theory of computation really is a lovely area, where accessible Big Results come thick and fast.

7.3 Formal arithmetics

(a) But now we must turn to formal theories of arithmetic.

We standardly focus on *First-order Peano Arithmetic*, PA. It will be no surprise to hear that this theory has a first-order language and logic. It has a built-in constant 0 to denote zero, has symbols for the successor, addition and multiplication functions (to keep things looking familiar, we still use a prefixed S, and infix + and $\times$). Its quantifiers run over the natural numbers. We can form the sequence of numerals 0, S0, SS0, SSS0, ... And we will use $\bar{n}$ to abbreviate the numeral obtained by writing n occurrences of S before 0, so $\bar{n}$ denotes n .

PA has the following three pairs of axioms governing the three built-in functions:

$$\begin{aligned} \forall x \ 0 \neq Sx \\ \forall x \forall y (Sx = Sy \rightarrow x = y) \\ \forall x \ x + 0 = x \\ \forall x \forall y \ x + Sy = S(x + y) \\ \forall x \ x \times 0 = 0 \\ \forall x \forall y \ x \times Sy = (x \times y) + x \end{aligned}$$

The first pair of axioms says that zero is not the successor of any number, and that distinct numbers have distinct successors. Thus the numerals behave as intended: zero, its successor, the successor of its successor, and so on are all distinct. The other

two pairs of axioms formalize the equations defining addition and multiplication which we have met before.

And then, crucially, there is also an arithmetical induction principle. As noted in §5.2, in a first-order framework we can stipulate that

Any wff of the form $(A(0) \wedge \forall x(A(x) \rightarrow A(Sx))) \rightarrow \forall xA(x)$ is an axiom,

where $A()$ stands for some suitable formula. Alternatively, one can present induction as an inference rule:

From $A(0)$ and $\forall x(A(x) \rightarrow A(Sx))$ we can infer $\forall xA(x)$.

You need to get some elementary familiarity with the resulting theory.

(b) But why concentrate on first-order PA? We've emphasized in §5.2 that our informal induction principle is most naturally construed as involving a second-order generalization – *for any arithmetical property P* , if zero has P , and if a number which has P always passes it on to its successor, then every number has P . And when Richard Dedekind (1888) and Giuseppe Peano (1889) gave their axioms for what we can call Dedekind-Peano arithmetic, they correspondingly gave a second-order formulation for their versions of the induction principle. Put it this way: the Dedekind-Peano principles quantify over *all* properties of numbers, while in first-order PA our induction principle rather strikingly only deals with those properties of numbers which can be expressed in the first-order language of PA. Why go for the weaker first-order principle?

Well, we have already addressed this in Chapter 5: first-order logic is much better behaved than second-order logic. And some would say that second-order logic is really just a bit of set theory in disguise. So, the argument goes, if we want a theory of pure arithmetic, one whose logic can be nicely formalized, we should stick to a first-order formulation just quantifying over numbers. Then something like PA's inference rule for induction (or the suite of axioms of the form we described) is the best we can do.

But still, even if we have decided to stick to a first-order theory, why restrict ourselves to the impoverished resources of PA, with only three function-expressions built into its language? For example, why not have an expression for the exponential function as well, and add to the theory the two defining axioms for that function? Indeed, why not add expressions for other recursive functions too, and then also include appropriate axioms for *them* in our formal theory?

(c) Good question. The answer is to be found in a very neat technical observation first made by Gödel. Once we have successor, addition and multiplication available, plus the usual first-order logical apparatus, we can in fact *already* express any other computable (i.e. recursive) function. To take the simplest sort of case, suppose f is a one-place recursive function: then there will be a two-place formula of PA's language which we can abbreviate $F(,)$ such that $F(\bar{m}, \bar{n})$ is true if and only if $f(m) = n$.

Moreover – a stronger claim – when $f(m) = n$, PA can *prove* $F(\overline{m}, \overline{n})$, and when $f(m) \neq n$, PA can *prove* $\neg F(\overline{m}, \overline{n})$. In this way, PA as it were already has the resources to ‘represent’ all the recursive functions, and prove the corresponding correct numerical instances. Similarly, PA can already represent any algorithmically decidable relation.

So PA is a *lot* richer than you might initially suppose. And it turns out that even an induction-free subsystem of PA known as Robinson Arithmetic (often called simply Q) can represent the recursive functions.

And this key fact puts you in a position to link up your investigations of PA with what you know about computability. For example, we quickly get a fairly straightforward proof that there is no mechanical procedure that a computer could implement to decide whether a given sentence in the language of PA is provable in the theory.

(d) On the other hand, despite its richness, PA is a first-order theory with infinite models, so – applying results from elementary model theory (see the previous chapter) – it is very easy to show that this first-order arithmetic will have non-standard models. The theory isn’t categorical. Rather, in addition to its intended standard model, it also has models whose domains contain more than a zero and its successors.

It is worth knowing at an early stage something about what some of these non-standard models can look like (they have a copy of the natural numbers in their domains but also additional ‘non-standard numbers’). And you will also want to further investigate the contrast with those second-order versions of arithmetic which *are* categorical (which don’t have non-standard models). This story soon becomes intriguingly complex: but do at least begin to explore it.

7.4 Towards Gödelian incompleteness

(a) Now for our third related topic: Gödel’s epoch-making incompleteness theorems. We’ll concentrate on the first of the two theorems here.

First-order PA, we said, turns out to be a very rich theory. But is it rich enough to settle every question that can be raised in its language? No! In 1931, Kurt Gödel proved that a theory like PA must be *negation-incomplete* – meaning that we can form a sentence G in its language such that PA proves neither G nor $\neg G$. How does Gödel do the trick?

(b) Let me give an outline sketch, which I hope will leave you wanting to find out more! So here goes:

- G1. Gödel introduces a *Gödel-numbering scheme* for a formal theory like PA, which is a way of systematically coding expressions of PA – and also sequences of expressions of PA – using natural numbers. The code number for an expression (or a sequence of expressions) is its unique *Gödel number*.

- G2. We can then define relations like Prf , where $Prf(m, n)$ holds if and only if m is the Gödel number of a PA-proof of the sentence with code number n . So Prf is a numerical relation which, so to speak, ‘arithmetizes’ the syntactic relation between a PA-proofs and their conclusions. (True enough, this isn’t kind of arithmetic relation we are familiar with. But it is perfectly well-defined.)
- G3. There’s a procedure for computing, given numbers m and n , whether $Prf(m, n)$ holds. Informally, we just decode m (that’s an algorithmic procedure). Then we check whether the resulting sequence of expressions – if there is one – is a genuine PA-proof, correctly constructed according to the rules of the game (proof-checking is another algorithmic procedure). If that sequence is a proof, check whether it ends with a sentence with the code number n (that’s another algorithmic procedure).
- G4. Since PA can express any algorithmically decidable *relation*, there will in particular be a formula in the language of PA which we can abbreviate Prf which expresses the effectively decidable relation Prf . This means that $Prf(\bar{m}, \bar{n})$ is true if and only if m codes a PA proof of the sentence with Gödel number n .
- G5. Now define $Prov(y)$ to be the expression $\exists x Prf(x, y)$. Then $Prov(\bar{n})$, i.e. $\exists x Prf(x, \bar{n})$, is true if and only if some number is the Gödel number of a PA-proof of the wff with Gödel number n . So $Prov(\bar{n})$ is true just if the wff with code number n is a theorem of PA. Therefore $Prov$ is naturally called a *provability predicate*.
- G6. Next, with only a little bit of cunning, we construct a *Gödel sentence* G in the language of PA with the following property: G is true if and only if $\neg Prov(\bar{g})$ is true, where $\bar{g}$ is the numeral for g , the code number of G .
- Don’t worry for the moment about how we do this construction (it involves a so-called ‘diagonalization’ trick which is surprisingly easy). Just note that G is true on the standard interpretation of PA’s language if and only if the sentence with Gödel number g is not a PA-theorem, i.e. if and only if G is not a PA-theorem.
- In short, G is true if and only if it isn’t a PA-theorem. So, stretching a point, it is rather as if G ‘says’ *I am unprovable in PA*.
- G7. Now, suppose G is a PA theorem. Then, since G is true if and only if it isn’t a theorem of PA, G would be false. So PA would have a false theorem. Hence, assuming PA is *sound* and only has true theorems (a reasonable assumption!), then it can’t prove G .
- Hence, since it is not provable, G is indeed true. Which means that $\neg G$ is false. Hence, still assuming PA is sound, it can’t prove $\neg G$ either.
- So, in sum, assuming PA is sound, it can prove neither G nor $\neg G$. As announced, PA is negation-incomplete.

Wonderful!

(c) Now the argument generalizes to other nicely axiomatized sound theories T which can express enough arithmetic (where being nicely axiomatized implies that we can effectively check whether a sequence of formulas really is a T -proof). We can use the same sort of cunning construction to find a true G_T such that T can prove neither G_T nor $\neg G_T$. But let's be really clear: this doesn't, repeat *doesn't*, say that G_T is 'absolutely unprovable', whatever that could mean. It just says that G_T and its negation are unprovable in T .

Ok, you might well ask, why don't we simply 'repair the gap' in T by adding the true sentence G_T as a new axiom? Well, consider the theory $U = T + G_T$ (to use an obvious notation). Then (i) U is still sound, since the old T -axioms are true and the added new axiom is true. (ii) U is still a nicely axiomatized formal theory, given that T is. (iii) U can still express enough arithmetic. So we can find a sentence G_U such that U can prove neither G_U nor $\neg G_U$.

And so it goes. Keep throwing more and more additional true axioms at T and our theory will remain negation-incomplete (unless it stops counting as being nicely axiomatized). So here's the key take-away message: any sound nicely axiomatized theory T which can express enough arithmetic will not just be incomplete but in a good sense T will be *incompletable*.

(d) Returning to PA, it is true that we haven't quite arrived yet at what's usually called the First Incompleteness Theorem for PA. For that, we need the extra step Gödel took: we drop the *semantic* assumption that PA is a sound theory whose theorems are true for a weaker syntactic *consistency* requirement.³ But let me at this point leave you to explore the (not very difficult) details. You will also want to find out about the Second Incompleteness Theorem and its significance. So, it really is time to start reading in earnest!

7.5 Main recommendations on arithmetic, etc.

(a) Those overviews in §§7.2 to 7.4 were brisk, but I hope they were more than enough to pique your interest. But if you want a somewhat more expansive introduction to the same territory, then you can very usefully look at one of the following:

1. Robert Rogers, *Mathematical Logic and Formalized Theories* (North-Holland, 1971), Chapter VIII, 'Incompleteness, Undecidability' (still quite discursive, extremely clear).
2. Robert S. Wolf, *A Tour Through Mathematical Logic* (Mathematical Association of America, 2005), Chapter 3, 'Recursion theory and computability';

³ But note, if we are going to weaken one assumption, we will have to strengthen another. Instead of merely requiring that the formula Prf expresses the relation Prf , we will need the stronger result that Prf represents Prf : whenever $\text{Prf}(m, n)$ holds, PA proves $\text{Prf}(\bar{m}, \bar{n})$.

and Chapter 4, ‘Gödel’s incompleteness theorems’ (more detailed, requiring more of the reader, though some students do really like this book).

(b) Turning to textbooks, how should we approach the area? Gödel’s 1931 proof of his incompleteness theorem actually needs only the machinery of primitive recursive functions. As we noted, these functions are only a subclass of the effectively computable numerical functions. A more general treatment of computable functions was developed a few years later (by Gödel, Kleene, Turing and others), and this in turn throws more light on the incompleteness phenomenon.

So there’s a choice to be made. Do you look at things in roughly the historical order, first introducing just the primitive recursive functions, explaining how they get represented in theories of formal arithmetic, and learning how to prove initial versions of Gödel’s incompleteness theorem – and *then* move on to deal with the general theory of computable functions? Or do you explore the general theory of computation first, only turning to the incompleteness theorems later? Most modern introductions take the second approach, as in this first recommended book:

3. Richard Epstein and Walter Carnielli, *Computability: Computable Functions, Logic, and the Foundations of Mathematics* (Wadsworth, 2nd edn., 2000: Advanced Reasoning Forum, 3rd edn., 2008).

This is an excellent introductory book on the standard basics, particularly clearly and attractively written. Part I, on ‘Fundamentals’, covers some background material, for example on the idea of countable sets (many readers will be able to speed-read through these initial chapters). Part II, on ‘Computable functions’, approaches computability in two ways: first via Turing-machine computability, and second via primitive recursive and then partial recursive functions, ending with a proof that the two approaches define the same class of effectively computable functions. Part III, ‘Logic and arithmetic’, turns to formal theories of arithmetic and the way that the representable functions in a formal arithmetic like Robinson’s Q or PA turn out to be the recursive ones. Formal arithmetics like Q and PA are then shown to be undecidable, and Gödelian incompleteness derived. The shorter Part IV has a chapter on Church’s Thesis (with more discussion than is often the case), and finally there is a chapter on constructive mathematics. There are many interesting historical asides along the way, and there is a very good historical appendix too.

By contrast, my own Gödel books take things in the more historical order:

4. Peter Smith, *An Introduction to Gödel’s Theorems** (2nd edition CUP, 2013: now downloadable from logicmatters.net/igt).

This starts by informally exploring various ideas such as effective computability, and then it proves two correspondingly informal versions of the first incompleteness theorem. The next part of the book gets down to work talking about formal arithmetics, developing some of the theory of primitive recursive functions, and explaining the ‘arithmetization of syntax’. Then it establishes more formal versions of Gödel’s first incompleteness theorem – carefully distinguishing ‘semantic’ and ‘syntactic’ versions. The book then discusses the second theorem and its significance.

The last part of *IGT* widens out the discussion to explore the idea of recursive functions more generally, discussing Turing machines and the Church-Turing thesis, and giving further proofs of incompleteness (e.g. deriving it from the undecidability of the halting problem for Turing machines).

Now, as is the way with these things, *IGT* grew much longer than the lecture notes on which it was based. So I did later put together a cut-down version,

5. Peter Smith, *Gödel Without (Too Many) Tears** (Logic Matters, 2020), which is also freely downloadable from logicmatters.net/igt.

This much shorter book may well give you all you need for a first foray into this area if your main initial interest is Gödelian incompleteness.

(c) Epstein and Carnielli’s text and my *IGT* should be very accessible to those without much mathematical background. Even more experienced mathematicians should appreciate the expansive and careful introductory orientation which they provide. But now let’s take half-a-step up in mathematical sophistication, to arrive at three classic books:

6. Herbert E. Enderton, *Computability Theory: An Introduction to Recursion Theory* (Associated Press, 2011).

This shorter book is written with attractive zip and lightness of touch (this seems a notably more relaxed book than Enderton’s earlier *Logic*). The first chapter is on the informal Computability Concept. There are then chapters on general recursive functions and on computing on register machines (where these have a more realistic architecture than Turing machines). It is then shown that the register-computable functions are exactly the recursive ones.

Chapter 4 is on the key idea of recursive enumerability. Chapter 5 makes ‘Connections to logic’ (including proving Tarski’s theorem on the undefinability of arithmetical truth and a semantic incompleteness theorem). The final two chapters push on to say something about ‘Degrees of unsolvability’ and ‘Polynomial-time computability’.

All this is very nicely and accessibly done.

This book, then, makes an excellent alternative to Epstein & Carnielli in particular. It is, however, a little more abstract and conventional in style – which is why I have on balance recommended E&C for many readers, though the more mathematical might well prefer Enderton.⁴

Next a justly famous and quite delightful book:

7. George Boolos and Richard Jeffrey, *Computability and Logic* (CUP 3rd edn. 1990).

A distinctive modern classic, wonderfully lucid and engaging, admired by generations of readers. It starts with an exploration of Turing machines, ‘abacus computable’ functions, and recursive functions (showing that different definitions of computability end up characterizing the same class of functions). And then it moves on to discuss logic and formal arithmetic, with a number of interesting discussions ranging some way beyond what is covered in *IGT* or by E&C.

There are in fact two later editions of Boolos and Jeffrey – heavily revised and considerably expanded – with John Burgess as a third author. But I know that I am not the only one to think that these later versions (good though they are) do lose something of the original book’s famed elegance and individuality and distinctive flavour. Still, whichever edition comes to hand, do read it!

Finally in this section, we come to an absolutely stand-out book that you should certainly tackle at some point (if not immediately). However, although it starts from scratch, I rather suspect that many readers will appreciate this book much more if they come to it *after* reading one or more of the previous recommendations in this section, which is why I only list it now:

8. Raymond Smullyan, *Gödel’s Incompleteness Theorems*, Oxford Logic Guides 19 (Clarendon Press, 1992).

This is delightfully short – under 140 pages – proving some rather beautiful, slightly abstract, versions of the incompleteness theorems (providing a lot of insight into what it takes to prove the various versions).

⁴ By the way, I should mention here that Chapter 3 of Enderton’s earlier *A Mathematical Introduction to Logic* (recommended in §4.2) gives a brisk treatment of different strengths of formal theories of arithmetic, and then proves the incompleteness theorem first for a formal arithmetic with exponentiation and then – after touching on other issues – shows how to use the β -function trick to extend the theorem to apply to arithmetic without exponentiation. Not the best place to start, I’d say, but that chapter could be very useful revision material.

This is another modern classic which anyone with a taste for mathematical elegance will find extremely rewarding.

(d) One comment: none of these books – including my longer one – gives a full proof of Gödel’s Second Incompleteness Theorem. The guiding idea is easy enough, but there is tedious work to be done in implementing it. If you *really* want more details, see the suggestions in §13.3.

7.6 Some parallel/additional reading

First, some books whose main emphasis is on Gödel’s theorems and related results; then a few books about computability more generally.

(a) I should start by mentioning an elementary book which vigorously debunks various myths about the wider significance of Gödelian incompleteness:

1. Torkel Franzén, *Gödel’s Theorem: An Incomplete Guide to its Use and Abuse* (A. K. Peters, 2005).

John Dawson, one of the co-editors of Gödel’s *Collected Works*, writes “Among the many expositions of Gödel’s incompleteness theorems written for non-specialists, this book stands apart. With exceptional clarity, Franzén gives careful, non-technical explanations both of what those theorems say and, more importantly, what they do not. No other book aims, as his does, to address in detail the misunderstandings and abuses of the incompleteness theorems that are so rife in popular discussions of their significance. As an antidote to the many spurious appeals to incompleteness in theological, anti-mechanist and post-modernist debates, it is a valuable addition to the literature.” Invaluable, in fact!

The same author has also written his own distinctive account of Gödelian incompleteness and related matters:

2. Torkel Franzén, *Inexhaustibility: A Non-Exhaustive Treatment* (Association for Symbolic Logic/A. K. Peters, 2004).

The first dozen chapters provide a very readable discussion of interwoven topics in logic, arithmetic, computability and incompleteness, which nicely complements more conventional treatments. Franzén also says more than is usual about ordinals, preparing the way for proof-theoretic applications (a topic that will concern us later).

(b) Next, the following excellent book could well have been an alternative main recommendation:

3. Christopher Leary and Lars Kristiansen, *A Friendly Introduction to Mathematical Logic** (2nd edn., Milne Library 2015), tinyurl.com/friendlylogic.

Chapters 4 to 7 give a very illuminating double treatment of matters related to incompleteness (and note, you don't have to have read the previous chapters in this book to follow the later ones, apart from noting the arithmetical system N introduced in their §2.8).

- i. L&K's first approach doesn't proceed overtly via computability. Instead of showing that certain syntactic properties are primitive recursive and then showing that all primitive recursive properties can be 'represented' in theories like N (as I do in *IGT*), L&K rely on directly showing that some key syntactic properties can be represented. This representation result then neatly leads to, inter alia, the incompleteness theorem.
- ii. L&K follow this, however, with a more general discussion of computability, and then use the introductory results they obtain to prove various further theorems, including incompleteness again.

This is all presented with the same admirable clarity as the first part of their book on FOL.

I should perhaps also mention another introductory text that is free to download from the Open Logic Project:

4. Richard Zach, *Incompleteness and Computability: An Open Introduction to Gödel's Theorems**, tinyurl.com/icompe-open.

Chapters 1 to 5 are on computability, representability in Q , and Gödelian incompleteness: so these chapters cover a good deal in just 120 sparsely printed pages (formatted for onscreen reading). The presentation is clear but can be compressed and symbol-heavy. Some readers might nevertheless find these chapters a manageable first introduction to their topics. For many readers, however, they will serve better in a different role, as useful revision material.

The remaining chapters continue the discussion of computability, consider models of formal arithmetics, briskly discuss second-order logic, and introduce the lambda calculus. But again, as first introductions to their topics, these are likely to be too brisk for many.

- (c) To introduce the next book, the first thing to say is that it presupposes *very* little knowledge about sets, despite its title. Do just dive in!

5. Melvin Fitting, *Incompleteness in the Land of Sets** (College Publications, 2007).

This is a very engaging read, approaching the incompleteness theorem and related results in an unusual but highly illuminating way. From the book's

blurb: “Russell’s paradox arises when we consider those sets that do not belong to themselves. The collection of such sets cannot constitute a set. Step back a bit. Logical formulas define sets (in a standard model). Formulas, being mathematical objects, can be thought of as sets themselves – mathematics reduces to set theory. Consider those formulas that do not belong to the set they define. The collection of such formulas is not definable by a formula, by the same argument that Russell used. This quickly gives Tarski’s result on the undefinability of truth. Variations on the same idea yield the famous results of Gödel, Church, Rosser, and Post.”

(d) And finally on incompleteness – if only because I’ve been asked about the book so often – I suppose I should finish by mentioning the (in)famous

6. Douglas Hofstadter, *Gödel, Escher, Bach** (Penguin, 1979).

When students enquire about this, I helpfully say that it is the sort of book that you will probably really like if you like this kind of book, and you won’t if you don’t. It is, to say the very least, quirky, idiosyncratic and entirely distinctive.

However, as far as I recall, the parts of the book that touch on technical logical matters (and on Gödelian incompleteness in particular) are in fact quite reliable and won’t lead you astray.

(e) Now to turn to texts which are about computability more generally.

There are two short books in the American Mathematical Society’s ‘Student Mathematical Library’,

7. A. Shen and N. K. Vereshchagin, *Computable Functions* (AMS, 2003).

8. Rebecca Weber, *Computability Theory* (AMS, 2012).

S&V’s text is an elegant little book, which offers a differently structured quick tour through some of the Big Ideas. Best read, though, as a follow-up to more conventional texts.

The first half of Weber’s text again gives a readable introduction to some of the Big Ideas. But the second half moves rather too rapidly into more advanced topics (such as ‘finite injury priority arguments’ for the Friedberg–Muchnik theorem). Again, best read after more conventional texts.

So, setting those two books aside, what I do suggest to read next on computability, pushing on from the introductory texts listed in §7.5? First

9. Nigel Cutland, *Computability: An Introduction to Recursive Function Theory* (CUP 1980).

This is a deservedly much-reprinted classic, quite beautifully lucid and well-organized.

This *does* have the look-and-feel of a traditional maths textbook of its time (so perhaps with fewer of the classroom asides that we find in some modern, more discursive, books). However, if you got through most of Boolos and Jeffrey without too much difficulty, you ought certainly to be able to tackle Cutland as the next step. Very warmly recommended.

And of more recent books covering computability at this level, I particularly like

10. S. Barry Cooper, *Computability Theory* (Chapman & Hall/CRC 2003).

A very nicely done modern textbook. Read at least Part I of the book (about the same level of sophistication as Cutland, but with some extra topics), and then you can press on as far as your curiosity takes you, reaching excitements like the Friedberg-Muchnik theorem.

But now we are beginning to trespass on the territory of §13.3 which explores advanced texts on arithmetic and computability. So I'll draw a somewhat arbitrary line at this point.

7.7 A little history

- (a) If you haven't already done so, *do* read

1. Richard Epstein, 'Computability and undecidability – a timeline', printed at the very end of Epstein & Carnielli, listed in §7.5.

This will briskly give you the headline news you initially need. As you will see, 1936 was an *annus mirabilis*, with Church, Kleene, Turing and Post all proposing different but equivalent accounts of effective computability. For an illuminating discussion, see

2. Robin Gandy, 'The confluence of ideas in 1936' in R. Herken, ed., *The Universal Turing Machine: A Half-century Survey* (OUP 1988).

And if you want to look at some of the original papers which made up this remarkable episode, see the reprints in

3. Martin Davis, ed., *The Undecidable: Basic Papers on Undecidable Propositions, Unsolvability Problems and Computable Functions* (Raven Press 1965; Dover Publications 2004).

- (b) The Davis collection includes a translation of Gödel's original 1931 paper. But up to a few years ago, I would not have recommended that relative beginners seriously tackle this. However, help is now at hand from

4. Hal Prince, *The Annotated Gödel: A Reader's Guide to His Classic Paper on Logic and Incompleteness** (Homebred Press 2022).

5. Dirk W. Hoffmann, *Gödel's Incompleteness Theorems: A Guided Tour Through Kurt Gödel's Historic Proof* (Springer 2024).

Prince has retranslated Gödel's 1931 incompleteness paper in a somewhat more relaxed style than the versions in Davis or in the *Collected Works*, and – much more importantly – he interleaves admirably lucid commentary and explanations intended for those with relatively little prior exposure to logic. If you have already read something like my *IGT*, you should certainly be able to work through the annotated paper with Prince's help.

Hoffmann's book – originally published in German in 2017 – is about twice the length of Prince's, not least because there is more background scene-setting which may well be unnecessary for typical readers (for example, primers on logic and set theory). I prefer Prince's more focused approach.

You might also enjoy and learn a lot from

6. Charles Petzold, *The Annotated Turing* (Wiley, 2008)

An intriguing mix of historical context and an extensively annotated exposition of Turing's great 1936 paper 'On Computable Numbers . . . '.

(c) Finally, I can recommend

7. John Dawson, *Logical Dilemmas: The Life and Work of Kurt Gödel* (A. K. Peters, 1997).

This is, perhaps, not as lively as the Fefermans' biography of Tarski which I mentioned in §6.4 – but then Gödel was such a very different man. Dawson's more conservative biography is still fascinating and illuminating!⁵

⁵ You won't, however, get any logical insights from Stephen Boudiansky *Journey to the Edge of Reason: The Life of Kurt Gödel*, OUP 2021.